

هندسة المتطلبات، إدارة العمل الجماعي، وتأسيس بيئة Git

المحاضرة العملية الأولى - هندسة البرمجيات

📅 العام الجامعي 2026/2027

👤 إعداد وتقديم: المهندس ساهر الهمداني

 User Story

يصفني مستخدم
أريد تسجيل الدخول إلى النظام
حتى أتمكن من الوصول إلى
الوظائف المتاحة لي

Acceptance Criteria

- ✓ إدخال اسم المستخدم صحيح
- ✓ إدخال كلمة المرور صحيح
- ✓ عرض رسالة خطأ فشل الدخول



رسالة المحاضرة

من المبرمج إلى مهندس البرمجيات



لا تكتب الكود فقط، بل تبني نظاماً قابلاً للصيانة والتوسع.



نفهم المشكلة قبل اقتراح الحل.



نحوّل الأفكار إلى متطلبات واضحة وقابلة للتنفيذ.



نعمل ضمن فريق وأدوات احترافية مثل Git و GitHub

“

مهندس البرمجيات لا يسأل: كيف أكتب الكود فقط؟
بل يسأل أيضاً: لماذا أبني؟ وكيف نُختبر؟



نظام قابل للصيانة والتوسع



- ✓ تحليل المتطلبات
- ✓ تصميم معماري
- ✓ التطوير
- ✓ الاختبار
- ✓ النشر
- ✓ المراقبة والتحسين



الخيمة أم ناطحة السحاب؟

تشبيه يوضح لماذا نحتاج الهندسة



الخيمة



بناء سريع



شخص أو شخصان



تعديل مباشر



أخطاء أقل تكلفة

VS



ناطحة السحاب



تخطيط ومخططات



فريق متعدد التخصصات



مراجعات وضبط جودة



أي خطأ مكلف جداً



كلما كبر النظام، زادت الحاجة إلى المتطلبات والتنسيق والمراجعة.



ما الذي يحدث قبل كتابة الكود؟

سير العمل قبل مرحلة التنفيذ



البدء بالكود مباشرة يُضاعف إعادة العمل لاحقاً.

هندسة المتطلبات ببساطة

ما هو المتطلب؟ ولماذا هو مهم؟



المتطلب:

وصف واضح لشيء يجب أن يقدمه النظام أو شرط يجب أن يحققه.

لماذا هي مهمة؟



تمنع بناء
الحل الخاطئ



توحد
فهم الفريق



تسهّل الاختبار
والتسليم



تقلّل
إعادة العمل



مثال: يسمح النظام للطالب
برفع واجب إلكتروني.

REQUIREMENTS SPECIFICATION



- ✓ واضح
- ✓ قابل للقياس
- ✓ قابل للتحقق
- ✓ ضروري



المتطلبات الوظيفية وغير الوظيفية

ماذا يفعل النظام؟ وكيف يجب أن يعمل؟



المتطلبات الوظيفية

تصف ما الذي يجب أن يفعله النظام.

- تسجيل الدخول
- إضافة مهمة
- تعديل مهمة
- حذف مهمة
- تغيير حالة المهمة



المتطلبات غير الوظيفية

تصف جودة النظام أو القيود التي يعمل ضمنها.

- زمن الاستجابة أقل من ثانيتين
- حماية بيانات المستخدم
- يعمل على الهاتف والحاسوب
- سهولة الاستخدام
- قابلية التوسع



غير الوظيفية = كيف يجب أن يعمل؟



الوظيفية = ماذا يفعل النظام؟



مثال

في نظام إدارة المهام: إنشاء مهمة متطلب وظيفي، وسرعة الاستجابة متطلب غير وظيفي.

وثيقة SRS المصغرة

كيف نلخص المتطلبات قبل بدء التنفيذ؟

SRS وثيقة المتطلبات

01

02

03

04

05

06

01



وصف المشكلة

شرح واضح للمشكلة
السياق، والأسباب، والأثر.

02



الحل المقترح

عرض الحل المقترح وكيفية
معالجة المشكلة وتحقيق الأهداف.

03



المستخدمون
المستهدفون

تحديد الفئات المستفيدة
من النظام واحتياجاتهم.

04



نطاق المشروع

تحديد ما يشمله النظام
وما هو خارج النطاق.

05



المتطلبات
الوظيفية

الوظائف والخصائص التي
يجب أن يوفرها النظام.

06



المتطلبات غير
الوظيفية

الأداء، الأمان، القابلية للاستخدام،
الاعتمادية، وغيرها.



SRS = Software Requirements Specification

وثيقة تصف المشكلة، الحل، النطاق،
والمتطلبات الأساسية للنظام.

لماذا نستخدمها؟



توحد
فهم الفريق



تقلّل
الغموض



تسهّل التقسيم
إلى مهام



تدعم
التسليم المنظم



ابدأ بنسخة صغيرة، واضحة، وقابلة للتحديث.



العام الجامعي 2026/2027



إعداد وتقديم: المهندس ساهر الهمداني



07

محور التعلم العملي

قصص المستخدم

صيغة المتطلبات من وجهة نظر المستخدم



بصفتي [نوع المستخدم]



أريد [الوظيفة]



حتى أتمكن من [الفائدة]

مثال احترافي

“

بصفتي طالباً، أريد إضافة موعد نهائي للمهمة، حتى أتمكن من ترتيب أولويات الواجبات.

”

• خصائص القصة الجيدة •



قابلة للنقاش
والاختبار



قصيرة
وواضحة



توضح
القيمة



تركز على
المستخدم

لا تكتب التقنية بدلاً من الحاجة.



العام الجامعي 2026/2027



08

محور التعلم العملي

معايير القبول

كيف نعرف أن المهمة اكتملت فعلاً؟



Given

الوضع الابتدائي



When

الإجراء الذي
ينفذه المستخدم



Then

النتيجة المتوقعة

مثال



Given

أن الطالب
سجل الدخول



When

يدخل عنوان المهمة
ويضغط حفظ



Then

تظهر المهمة في
قائمة المهام

معايير قبول إضافية



حقل العنوان مطلوب



تظهر رسالة خطأ عند الإدخال الفارغ



لا يتم حفظ المهمة مرتين



تظهر رسالة نجاح بعد الإضافة



معايير القبول تجعل المطلوب قابلاً للاختبار.



العام الجامعي 2026/2027



إعداد وتقديم: المهندس سامر الهمداني

09

محور التعلم العملي

حالات الحافة

التفكير في السيناريوهات غير الطبيعية



البيانات

- حقل فارغ
- بيانات مكررة
- نص طويل جداً



الصلاحيات

- مستخدم غير مصرح
- الوصول لبيانات
- مستخدم آخر



الوقت والتاريخ

- تاريخ في الماضي
- انتهاء الجلسة



الشبكة والنظام

- انقطاع الاتصال
- قشل قاعدة البيانات
- ضغط الزر مرتين



المسار الطبيعي **لا يكفي**؛ مهندس البرمجيات يفكر في ما قد يحدث **عند الفشل أيضاً**.

Edge Cases = مواقف عند الحدود أو خارج الاستخدام الطبيعي



العام الجامعي 2026/2027



إعداد وتقديم: المهندس سامر الهمداني



محور التعلم العملي 10

Backlog و Kanban Board

الفرق بين قائمة الأعمال ولوحة متابعة التنفيذ

Backlog

قائمة الأعمال (مرتبة حسب الأولوية)

- | | | |
|---|---------------------------|---------|
| 1 | نظام تسجيل الدخول | ميزة |
| 2 | إدارة المستخدمين | ميزة |
| 3 | تقرير الطلبات | ميزة |
| 4 | تذكيرات البريد الإلكتروني | تحسين |
| 5 | دعم الوضع المظلم | تحسين |
| 6 | تكمال الدفع الإلكتروني | مستقبلي |

قائمة بجميع الأعمال والخصائص والأفكار المطلوبة.

- ✓ أعمال مستقلة
- ✓ ترتيب حسب الأولوية
- ✓ لا يعني بدء التنفيذ
- ✓ يدعم التخطيط

VS

Kanban Board



لوحة مرئية توضح حالة العمل الحالي.

- ✓ تعرض سير العمل
- ✓ توضح حالة كل مهمة
- ✓ تساعد في المتابعة
- ✓ تدعم العمل الجماعي



ماذا سننفذ؟ = Backlog

أين وصلت المهمة؟ = Kanban



Git و GitHub

إدارة الإصدارات والتعاون داخل الفريق



Git

نظام لإدارة الإصدارات وتتبع التغييرات في الملفات.

- ✓ يحفظ تاريخ التعديلات
- ✓ يدعم الفروع والدمج
- ✓ يساعد على الرجوع لإصدار سابق
- ✓ يقلل فقدان العمل



GitHub

منصة تستضيف مستودعات Git وتوفر أدوات التعاون.

- ✓ Issues
- ✓ Pull Requests
- ✓ Code Review
- ✓ Projects



Issue



Branch



Commit



Push



Pull Request



Review



Merge



Git يدير الإصدارات، و GitHub ينظم التعاون بين أعضاء الفريق.



العام الجامعي 2026/2027



إعداد وتقديم: المهندس ساهر الهمداني



12 محور التعلم العملي

دورة العمل باستخدام Git و GitHub

• من المتطلب إلى الدمج والمراجعة •



1 Issue



2 Branch



3 Edit / Change



4 Commit



5 Push



6 Pull Request



7 Review



8 Merge



9 Close Issue



Issue: تمثل المهمة أو المشكلة المطلوب تنفيذها



Branch: فرع مستقل لكل مهمة



Pull Request: بوابة المراجعة قبل الدمج



Review & Merge: ضمان الجودة ثم إدخال التغيير إلى main



كل مهمة تمر بمسار واضح: Issue → Branch → PR → Review → Merge



العام الجامعي 2026/2027



إعداد وتقديم: المهندس ساهر الهمداني



13 محور التعلم العملي

النشاط العملي داخل الفرق

خطوات التطبيق في المحاضرة العملية



الأدوار المقترحة

- Team Coordinator
تنسيق ومتابعة الفريق
- Requirements Owner
تحليل المتطلبات وتوثيقها
- Repository Maintainer
إدارة المستودع والإعدادات
- Developers
تنفيذ المهام والبرمجة
- Reviewer
مراجعة الجودة والدمج



الهدف ليس كتابة كود كثير، بل بناء طريقة عمل صحيحة داخل الفريق.

استخدام الذكاء الاصطناعي بشفافية

مسموح بشرط التوثيق في AI_Log.md



استخدامات مسموحة

- ✓ اقتراح حالات الحافة
- ✓ تحسين صياغة User Story
- ✓ مراجعة معايير القبول
- ✓ تحسين تنسيق Markdown
- ✓ شرح أوامر Git



استخدامات غير مقبولة

- ! النسخ دون فهم
- ! إنشاء متطلبات غير مفهومة
- ! التسليم دون توثيق
- ! الاعتماد الكامل على الأداة
- ! رفع بيانات حساسة



AI_Log.md

Date	Student	Tool	Purpose	Accepted Suggestions	Rejected Suggestions	Human Review
=====	=====	=====	=====	=====	=====	=====



الذكاء الاصطناعي **مساعد** في التفكير، وليس بديلا عن القرار الهندسي.



العام الجامعي 2026/2027



إعداد وتقديم: المهندس ساهر الهمداني



المخرجات والتقييم

• ما الذي سيتم تسليمه في نهاية المحاضرة؟ •



Deliverables

- ✓ منظم GitHub Repository
- ✓ README.md
- ✓ docs/SRS.md
- ✓ AI_Log.md
- ✓ Project Board
- ✓ الأقل على 5 Issues
- ✓ Branch + Commit + Push
- ✓ Pull Request + Review + Merge



التقييم

الوزن	المعيار
10	1 وضوح الفكرة
20	2 جودة Mini-SRS
15	3 User Stories & Criteria
10	4 Edge Cases
10	5 Issues & Board
10	6 Git Workflow
10	7 Pull Request Review
5	8 AI Log
10	9 العمل الجماعي والتنظيم
100	المجموع



المشروع الجيد لا يبدأ بالكود فقط، بل بمتطلبات واضحة وعمل جماعي منظم.



العام الجامعي 2026/2027



إعداد وتقديم: المهندس ساهر الهمداني



محور التعلم العملي 16